

Secure API Challenge: Expense Claims API

FIL Fresher Batch • Secure Software Engineering • Team competition

You will design and build three REST APIs for a corporate **expense-reimbursement** system (the kind of thing SAP Concur, Zoho Expense or an in-house HR portal does). The APIs must be:

1. **Correct.** Filtering, sorting, pagination and the approval workflow behave exactly as specified.
2. **Secure.** Authentication and authorisation are well designed, and the code follows the **OWASP Top 10:2025** practices you learned in class.
3. **Proven.** An automated judge suite of 436 black-box test cases checks your API. You get the judge only **after** the build phase (see below), so design and test against this specification.

You may use **any language and framework** (FastAPI, Spring Boot, ASP.NET Core, Express, ...). The judge talks to your service only over HTTP.

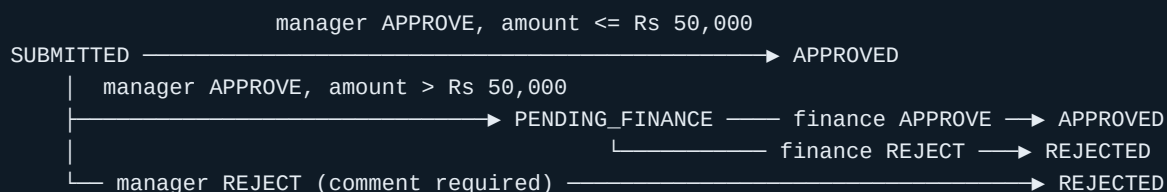
How the competition runs

Phase	What happens
1. Analyse & design	Read this spec, then prepare slides 1 (requirement analysis) and 2 (design decisions).
2. Build & self-test	Implement the 3 APIs. Write your own tests from this spec; no judge tests are shared yet.
3. Code freeze	Commit your final code. No code changes after this point.
4. Judge release	The trainer shares the judge suite (<code>judge/</code>). You run it against your frozen code: <code>python grade.py --team <name> .</code>
5. Record results	Copy the result matrix (total / passed / failed per category) into slide 3, and explain every failure.
6. Present	3 slides + Q&A. The trainer re-runs the judge on your frozen commit to confirm your numbers.

Out of scope for this exercise: concurrency and race conditions (parallel requests, locking, load). All judge requests are sent one at a time.

1. The business scenario

Employees spend money on company business (travel, meals, hotels, training) and file an **expense claim** to be reimbursed. Claims then go through an approval workflow:



APPROVED and **REJECTED** are **final**. The threshold is `approval_threshold_paise` in the seed file (5,000,000 paise = Rs 50,000). It is inclusive: exactly Rs 50,000 needs no finance step.

Roles

Role	Can see (list API)	Can decide (decision API)
EMPLOYEE	Own claims only	Nothing (403)
MANAGER	Own claims + claims of direct reports	SUBMITTED claims of direct reports only
FINANCE_ADMIN	All claims	PENDING_FINANCE claims only
AUDITOR	All claims (read-only)	Nothing (403)

Separation of duties: nobody may decide their **own** claim, whatever their role (`403 self_approval_forbidden`). A manager's manager does **not** inherit authority over the team below (no skip-level approvals). Claims of a disabled employee can still be decided by their manager.

Seed data (`seed/seed_data.json`)

Your service must **load the seed file on every start** from the path in `SEED_FILE` and begin from that state. Nothing needs to persist between restarts; an in-memory store or a database rebuilt at start-up are both fine.

- 10 users: `asha`, `ravi`, `neha`, `kiran`, `sunil`, `dinesh` (disabled) are employees; `priya` and `vikram` are managers; `farah` is finance admin; `meera` is auditor. `manager_id` defines the reporting line.
- 30 expense claims (ids 1001–1030) in every status and category.
- Passwords are in plain text **only so that the judge can log in**. Hash them when you load them (Argon2id, bcrypt or PBKDF2 ≥ 600k). Never store, return or log the plain text.

Reporting lines: `priya` manages `asha`, `ravi`, `dinesh`, `kiran`. `vikram` manages `neha`, `priya`, `farah`, `sunil`.

2. Runtime contract (how the judge starts your service)

The judge starts your process with a command you configure (`start_cmd` in `judge/judge_config.json` , which you fill in when the judge is released) and passes configuration **only through environment variables**. Build for this contract from day one:

Variable	Example	Rule
<code>APP_ENV</code>	<code>test / prod</code>	Only <code>test</code> or <code>prod</code> are valid. If it is unset, behave as <code>prod</code> . Any other value means refuse to start.
<code>APP_PORT</code>	<code>8765</code>	Listen on <code>127.0.0.1:APP_PORT</code> . <code>start_cmd</code> may also use the <code>{port}</code> placeholder.
<code>JWT_SECRET</code>	<code>48+ random chars</code>	Required, ≥ 32 bytes . No default and no fallback.
<code>JWT_ISSUER</code>	<code>expense-api</code>	Value of the <code>iss</code> claim. Default <code>expense-api</code> .
<code>ACCESS_TOKEN_TTL_SECONDS</code>	<code>900</code>	Token lifetime, 60–900. Default 900.
<code>CORS_ALLOWED_ORIGINS</code>	<code>https://expenses.example.com</code>	Comma-separated exact origins. <code>*</code> means refuse to start.

Variable	Example	Rule
LOGIN_RATE_LIMIT_PER_MINUTE	10	Per client IP, on the login API. Default 10.
SEED_FILE	/abs/path/seed_data.json	Seed data location.

Refuse to start means printing an error (without the secret) and exiting with a **non-zero** exit code within a few seconds. It applies when `JWT_SECRET` is missing or shorter than 32 bytes, when `CORS_ALLOWED_ORIGINS` contains `*`, or when `APP_ENV` is invalid.

In `prod`, interactive API docs and explorers (Swagger UI, `/docs`, `/openapi.json`, `/v3/api-docs`, actuator, ...) must **not** be served (return 401/403/404). In `test` they are optional.

3. Cross-cutting requirements (apply to every API)

3.1 Error envelope: one shape for every error

```
{ "error": { "code": "validation_error", "message": "page_size must be an integer between 1 and 100",
            "correlation_id": "3f0c1f0e-6a1b-4a53-9a55-1b8f8f7f2c11", "details": [ ... optional
... ] } }
```

- `correlation_id` equals the `X-Request-ID` response header.
- Messages are for humans and must never contain stack traces, exception class names, file paths, SQL or framework internals.

HTTP	code	When
400	validation_error	Any invalid input: body, query, path, header, malformed JSON
401	invalid_credentials	Login failed (every reason, same message)
401	not_authenticated	Missing / invalid / expired token, or unknown / disabled user. Add header <code>WWW-Authenticate: Bearer</code>
403	forbidden	Role not allowed
403	self_approval_forbidden	Deciding your own claim
404	not_found	Claim does not exist or is outside your scope (indistinguishable)
405	method_not_allowed	Wrong HTTP method on a known route
409	invalid_state	Claim is not in a state you may act on
409	idempotency_conflict	Idempotency-Key reused for a different request
413	payload_too_large	Body larger than 16 KB (16,384 bytes)
415	unsupported_media_type	POST body whose Content-Type is not <code>application/json</code>
429	rate_limited	Login throttled. Add a <code>Retry-After</code> header (seconds, 1-60)
500	internal_error	Unexpected failure: generic message only, details go to the log

Unknown routes return 404 (or 401) using the same envelope.

3.2 Security headers on every response (success and error)

X-Content-Type-Options: nosniff · X-Frame-Options: DENY · Content-Security-Policy: default-src 'none'; frame-ancestors 'none' · Referrer-Policy: no-referrer · Cache-Control: no-store. Also send Strict-Transport-Security (recommended, not tested over HTTP). Do **not** send X-Powered-By, and don't put a version number in the Server header.

3.3 CORS

Allow only the exact origins in CORS_ALLOWED_ORIGINS, and never *. Look-alike origins (https://expenses.example.com.evil.io, http://..., null) get no Access-Control-Allow-Origin.

3.4 Request tracing

Every response has an X-Request-ID header. If the client sent one matching ^[A-Za-z0-9-]{8,64}\$, echo it back. Otherwise generate a new one (UUID), and never reflect an invalid value.

3.5 Security logging (stdout, one JSON object per line)

Every event has at least ts, level, event and request_id (the same value as the response's X-Request-ID). Required events:

event	When	Useful fields
login_success	Token issued	user_id, username
login_failed	Any failed login	username, reason
account_locked	Lockout threshold reached	username
auth_failed	Bad/missing token on a protected API	reason, path
access_denied	Any 403, and 404s caused by scope	user_id, expense_id, reason
expense_decided	A state transition happened (once; not for idempotent replays)	user_id, expense_id, from_status, to_status

Never log passwords (correct or attempted), tokens or any part of them, the Authorization header, or JWT_SECRET. Non-JSON lines (framework banners) are allowed but ignored.

3.6 Non-functional requirements (sequential requests on the judge machine)

NFR	Target
List API latency (page_size=100 with filters)	p95 < 300 ms
Decision API latency	p95 < 300 ms
Login latency (slow hash included)	p95 < 1.5 s

4. API 1: POST /api/v1/auth/token (login)

Request (Content-Type: application/json):

```
{ "username": "asha", "password": "Monsoon-Chai-Walks-2026" }
```

Field	Rule
username	Required string, 1-64 chars, no control characters. Exact match.
password	Required string, 1-128 chars.
anything else	Unknown fields → 400 (e.g. "role": "FINANCE_ADMIN").

200 response, with header `Cache-Control: no-store`:

```
{ "access_token": "<JWT>", "token_type": "Bearer", "expires_in": 900 }
```

Token rules (JWT):

- Signed **HS256** with `JWT_SECRET`. Claims: `sub` (user id **as a string**, e.g. "1"), `iss`, `iat`, `exp`, and optionally `jti`. `exp - iat ≤ ACCESS_TOKEN_TTL_SECONDS`.
- Minimal claims**. No email, name, password, hash or other personal data. A `role` claim is allowed, but the server must **never trust it** (see 5.2).

Failures:

Situation	Response
Wrong password, unknown user, disabled user, locked account	401 invalid_credentials , byte-for-byte the same message in every case (no user enumeration, no "account locked" hint)
5 consecutive failed attempts for an existing username	Account locked for 15 minutes: even the correct password gets the generic 401. A successful login resets the counter. The lock affects only that account.
More than <code>LOGIN_RATE_LIMIT_PER_MINUTE</code> requests from one client IP in 60 s	429 <code>rate_limited</code> + <code>Retry-After</code> , even for correct credentials
Invalid body / JSON	400 · wrong Content-Type → 415 · > 16 KB → 413
Any method except POST	405

5. API 2: `GET /api/v1/expenses` (list with filters)

Requires `Authorization: Bearer <token>`. Tokens are accepted **only** from this header, never from the query string or cookies.

5.1 Query parameters (all optional)

Param	Format / rule	Meaning
status	Comma-separated list of <code>SUBMITTED</code> , <code>PENDING_FINANCE</code> , <code>APPROVED</code> , <code>REJECTED</code> (case-sensitive)	Status is one of the values
category	One of <code>TRAVEL</code> , <code>MEALS</code> , <code>LODGING</code> , <code>TRAINING</code> , <code>OFFICE_SUPPLIES</code> , <code>INTERNET</code>	Exact category
employee_id	Positive integer (1-10 digits)	Claims of this employee
from_date, to_date	Strict YYYY-MM-DD, real calendar date; <code>from_date ≤ to_date</code>	<code>expense_date</code> within range, both inclusive

Param	Format / rule	Meaning
<code>min_amount</code> , <code>max_amount</code>	Integer paise 0-100,000,000; <code>min ≤ max</code>	<code>amount_paise</code> within range, inclusive
<code>q</code>	1-50 chars, no control characters	Case-insensitive literal substring of <code>description</code> . <code>%</code> , <code>_</code> , <code>*</code> , <code>.</code> , <code>'</code> and <code>\</code> are ordinary characters, not wildcards.
<code>sort</code>	<code>expense_date</code> , <code>amount_paise</code> or <code>submitted_at</code> , optionally prefixed with <code>-</code> for descending. Default <code>-submitted_at</code> .	Ties are always broken by <code>id</code> ascending
<code>page</code>	1-10000, default 1	
<code>page_size</code>	1-100, default 20	

- Filters are combined with **AND**.
- **Unknown parameters and repeated parameters** → 400 (e.g. `?role=ADMIN, ?page=1&page=2`).
- Any other violation of the table → 400 `validation_error`, with a message that names the parameter.
- A page past the end returns 200 with `items: []` and the real `total`.

5.2 Authorisation: scope first, then filters

The server first computes the caller's **visible set** from the role table in §1, then applies the filters. Filters can only **narrow** that set and never widen it: asha asking `?employee_id=2` gets 200 with `total: 0`, not ravi's claims.

The caller's role and status always come from **your user store**, never from token claims. A correctly signed token whose `role` claim says `FINANCE_ADMIN` for asha still gives asha's scope.

5.3 Response 200

```
{
  "items": [
    { "id": 1002, "employee_id": 1, "category": "MEALS", "amount_paise": 85000, "currency": "INR",
      "expense_date": "2026-07-05", "description": "Team lunch: 50% off offer",
      "status": "SUBMITTED", "submitted_at": "2026-07-07T09:02:00Z" }
  ],
  "page": 1, "page_size": 20, "total": 8
}
```

Items contain **exactly** these nine fields and nothing more: no password hashes, emails or internal columns.

`amount_paise` is an integer. `submitted_at` is ISO-8601 in UTC. Text is returned as JSON data (`application/json`), never as HTML.

6. API 3: `POST /api/v1/expenses/{id}/decision` (approve / reject)

Headers: `Authorization: Bearer <token>`, `Content-Type: application/json`, and `Idempotency-Key: <8-64 chars of A-Z a-z 0-9 ->` (required).

Body:

```
{ "decision": "APPROVE" }
{ "decision": "REJECT", "comment": "Receipt is missing for this claim" }
```

Field	Rule
decision	Required, exactly APPROVE or REJECT (case-sensitive string)
comment	Optional string ≤ 500 chars; required for REJECT with ≥ 10 non-blank characters
anything else	400 (no mass assignment: status , amount_paise , employee_id , ... are rejected)

{id} must match `^[1-9][0-9]{0,9}$`. Otherwise → 400.

Evaluation order (the first failing check decides the response):

1. Token valid? → otherwise **401**
2. Role is `MANAGER` or `FINANCE_ADMIN`? → otherwise **403 forbidden** (before revealing anything about the claim)
3. Path id, Idempotency-Key, Content-Type, body valid? → otherwise **400 / 415 / 413**
4. Claim exists **and** is visible to the caller? → otherwise **404 not_found**
5. Caller is not the claim's owner? → otherwise **403 self_approval_forbidden**
6. Idempotency: the same caller already used this key → same request (same claim + same body) returns the **stored 200 response** with no second transition; a different request → **409 idempotency_conflict**. Keys are scoped per user, and only successful decisions are stored.
7. State allowed for this role (manager: `SUBMITTED`, finance: `PENDING_FINANCE`)? → otherwise **409 invalid_state**
8. Apply the transition → **200** with the updated claim, in the same nine-field shape as a list item.

7. What you submit

1. **Source code** in a Git repository with a README (how to run it, and `start_cmd`). Include a dependency lock file with exact versions (A03).
2. **Judge report** (after the judge is released): `judge/reports/judge_report.md` / `.html`, produced by `python grade.py --team <name>` against your frozen commit.
3. **Exactly 3 slides**, using `slides/Submission_Template.pptx`:
4. **Requirement analysis**: actors and roles, the three APIs, business rules, abuse cases / threat model (STRIDE), and your assumptions.
5. **Design decisions**: architecture, authN/authZ pipeline, data-scope strategy, validation strategy, idempotency approach, and an OWASP Top 10 mapping with the trade-offs you made.
6. **Judge test case results**: the result matrix (total / passed / failed per category and overall), each failure with its root cause, how you would fix it, and what you would do next. Fixes are explained, not applied: the code stays frozen.
7. **Presentation**: 7 minutes per team, followed by 5 minutes of Q&A from the other teams and the trainer.

8. Scoring

Component	Weight
Automated judge (10 categories × 10 pts, scaled)	50%
Code review against the OWASP checklist (hashing, parameterised queries, secrets, dependencies, fail-closed)	20%
Slides: requirement analysis & design decisions	15%
Q&A: can every team member explain every decision?	15%

Judge categories: AUTHN · TOKEN · RBAC · OBJECT · FILTER · INPUT · WORKFLOW · CONFIG · LOGGING · RESILIENCE. The judge and its `README.md` are handed out at the judge-release step.

Fair-play rules. Don't edit `judge/` or `seed/`. Don't change code after the code freeze. Don't special-case the judge (for example by detecting test usernames or keys). Trainers re-run every frozen commit on a clean machine, and the numbers on slide 3 must match.